

Project Report

Teja Yeswanth.S and Sai Rishanth Reddy.S

April 29, 2026

Contents

1	External Libraries Used	2
2	Implemented Features	2
3	Code Logic	3
3.1	Generic Class	3
3.1.1	Tic Tac Toe	3
3.1.2	Connect 4	3
3.1.3	Othello	3
3.2	Authentication System	3
3.3	Leaderboard	4
4	Hurdles Faced and Solutions	4
5	Future Improvements	5
6	Bibliography	6

1 External Libraries Used

No external libraries were used in this project, only allowed modules were used.

2 Implemented Features

Some features apart from the ones which were asked are added, these include:

- Player can exit the login or registration process anytime in the terminal
- The placeholder of the coin gets highlighted when the player hovers over it in Othello
- Player's turn is displayed on the screen
- The coin assigned to each player is displayed on the screen
- Number of coins each player has is displayed on the screen in Othello
- Icons in the game menu gets highlighted when the player hovers over it
- Designed background images for each game and the game menu with suitable colour combinations

3 Code Logic

3.1 Generic Class

The base class has all the required variables defined.

3.1.1 Tic Tac Toe

10x10 grid with 5-in-a-row win condition. 'xxxxx' in `isstring` gives true or false this is used to check for win condition. To avoid using loops we checked only at the position where the player has just played. A function is defined to draw X shape.

3.1.2 Connect 4

7x7 grid with 4-in-a-row win condition. Same logic as tictactoe is used to check for win condition. Tictactoe is similar to connect4 in most of the code.

3.1.3 Othello

Each direction is checked for validity of move and then the coins co-ordinates are stored in a numpy array and then the coins are flipped by looking at the array. The `isvalidmove` function returns a string representing the direction in which the coins are to be flipped, which was later used in `updateboard-aftermove` function to update the numpy board. The winner is decided by counting the number of coins, which is done using `np.sum(board == player)` for each player and comparing the counts. Valid moves possible for the current player are drawn using `drawvalidmovespositions` function, which gets the list of co-ordinates of valid moves from `validmoves` function.

3.2 Authentication System

All authentication is taken care of by `main.sh`, when a username is entered it checks if the username exists in the `users.tsv`, if it does then it prompts for password and checks if the password is correct, if the username doesn't exist then it prompts for registration and adds the username and password to the `users.tsv` file. After certain entries there will be an option to stop the process running in the terminal by taking 'x' as input.

3.3 Leaderboard

The file `leaderboard.sh` takes the arguments from the graphical interface of winner display page after the game ends. It takes this element as a value to sort the data from the `history.csv` file. It reads the data and the winner-loser content from the `history.csv` file and by using the `awk` command it analyses and calculates the wins of losses of each player in each game by reading each line of `history.csv`. It then sorts this analysed data with respect to the value taken as argument i.e wins ,losses or win/loss ratio and displays the sorted data in the terminal with the header line being highlighted. This file is called after every game ends.

4 Hurdles Faced and Solutions

- Designing the UI using `pygame` was a bit difficult, but we overcame it by looking at various designs and implementing the one which we liked the most.
- Implementing the logic for Othello was a bit tricky, but we overcame it by breaking down the problem into smaller parts and solving each part separately.
- We faced a problem with circular imports in `BaseGame` class which was originally in `game.py`, we solved it by keeping the `BaseGame` class in `base.py` and importing it in the game files.
- After the end of each game, the data appended in the `history.csv` file has carriage values but we overcame it by using `gsub` command in the `leaderboard.sh` to remove the carriage values.

5 Future Improvements

As discussed in Section 3, if we had 10 hrs more time improvements we would consider are :

- More games
- Better UI with hover effects in all games
- Animation in connect4 (falling coins) and in tictactoe (drawing X and O and striking off the winning line)
- Tie conditions in all games and it's stats in the leaderboard
- Background music and sound effects for each game

6 Bibliography

- pygame documentation: <https://pyga.me/docs/>
- numpy documentation: <https://numpy.org/doc/2.4/index.html>
- Python documentation: <https://docs.python.org/3/>
- Matplotlib: <https://matplotlib.org/stable/index.html>